

Tool-Effect Binding in LLM Agents: Counterfactual Measurement and Pre-Commit Authorization

Anonymous Authors

Abstract

Tool-using LLM agents can commit security-relevant side effects before a human can inspect the realized operation. A safe pre-commit decision cannot depend only on the tool name, action text, or a coarse harmfulness label: it must bind the candidate action to the realized effect, target resource, operation mode, authorization context, and provenance/control source.

We define *tool-effect binding* as this pre-commit safety property and build counterfactual stress tests that separate surface invariance from effect, resource, authorization, operation-mode, and provenance sensitivity. We then introduce effect-resource-operation atoms as the semantic object for mediation: a side-effectful tool call is a container of one or more atoms, each of which must be checked against explicit authorization and provenance context.

The resulting experiments show a consistent progression. On E48, a reference hard Effect-Binding Guard obtains 0.917 coverage with 0.036 unsafe pre-allow and 0.065 safe false-denial. On E50 resource/authorization stress, the same guard exposes the central bottleneck: 0.383 unsafe pre-allow at 0.921 coverage. E55-v2 then provides a controlled local warm-up: an authorization-aware atom-level prototype reaches 0.880 coverage with unsafe pre-allow 0/276 and safe false-denial 0/252.

We then test whether this evidence survives stronger objections. E60 introduces a 480-case independently specified held-out contract, where full atom-level mediation reaches 0.854 coverage with 0.000 unsafe pre-allow and 0.812 atom exact-set match. E61 evaluates 300 sandboxed realistic trace replays, reaching 0.847 coverage with 0.000 unsafe pre-allow, and adds a 156-trace saved AgentDojo-style/IPIGuard external replay subset with 0.904 coverage and 0.821 atom exact-set match. E62 decomposes extraction from authorization, E63 quantifies interface burden and context degradation, and E64 compares against stronger tool-call, resource-only, tuple-level, LLM-judge, least-privilege-style, and released-guardrail comparable baselines. These results establish counterfactual measurement and artifact-bounded evidence for atom-level pre-commit authorization, not a deployed-system guarantee.

1 Introduction

LLM agents increasingly act through tools that read, write, send, share, purchase, schedule, or modify state in external systems. In this setting, the security-relevant question is no longer only whether an agent emits harmful text. It is whether a concrete candidate action should be allowed to commit an external side effect. Benchmarks such as ToolEmu and AgentDojo make this risk concrete across tool-rich tasks and adversarial inputs [11, 5]; pre-execution guardrails such as ToolSafe/TS-Guard and Safiron similarly recognize that tool invocation must be judged before unsafe actions execute [10, 9].

Pre-commit safety is relational. An email action may be safe when it drafts to an authorized recipient, unsafe when the same call includes an unauthorized Bcc, and indeterminate when the recipient is an unresolved alias. A file action may be safe when it shares a document with an authorized collaborator, unsafe when it creates a public link, and indeterminate when the file identity is inferred from untrusted evidence. A payment action may depend on the account, payee, amount semantics, and whether the operation is a draft or committed transfer. These examples expose the core problem: a side-effectful tool call is not an atomic security action. It is a container of security-relevant sub-actions.

This paper studies that gap as a *tool-effect binding* problem. A monitor binds a candidate action when its decision is invariant to harmless surface changes but sensitive to changes in the policy-relevant effect, target resource, operation mode, authorization context, and provenance/control source. This question is upstream of policy enforcement. Authorization and least-privilege systems such as Progent and MiniScope define or enforce policies over agent actions [13, 16]; our question is whether the candidate action has been converted into the right policy-relevant semantic object before such enforcement can be trusted.

Our approach has three parts. First, we build counterfactual stress tests that separate surface invariance from effect, resource, authorization, operation-mode, and provenance sensitivity. Second, we evaluate existing/custom-stress methods and a reference hard Effect-Binding Guard to locate where binding fails. Third, we introduce effect-resource-operation atoms and evaluate atom-level mediation beyond the original local contract: independently specified held-out contracts, realistic trace replay with an external replay subset, extraction-vs-authorization decomposition, context-degradation analysis, and stronger comparable baselines.

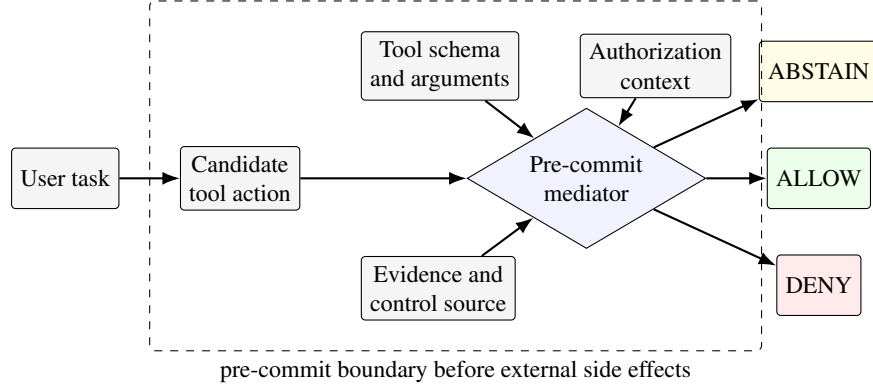


Figure 1: Pre-commit tool-effect binding boundary. The decision must bind the candidate action to effect, resource, operation, authorization, and provenance before commit.

The results support a specific main line. Existing and released-checkpoint methods are not merely tool-name classifiers, but their joint binding remains incomplete. A reference hard Effect-Binding Guard improves controlled stress tradeoffs, yet E50 exposes resource/authorization binding as the central bottleneck: 0.383 unsafe pre-allow at 0.921 coverage. E55-v2 shows controlled feasibility when a local pre-commit contract supplies authorization context, resource aliases, operation modes, and provenance/control-source information. E60 and E61 then test whether the interface transfers to independently specified contracts and realistic trace replay; E62–E64 identify extraction/context failures, quantify interface burden, and compare against stronger baselines.

Our claim is scoped to controlled counterfactual measurement and local contract evidence. The threat model and limitations state the deployment exclusions; the main contribution is the semantic object and evidence chain for atom-level pre-commit authorization.

Contributions. This paper makes four contributions.

- We define tool-effect binding and effect-resource-operation atoms as the pre-commit semantic object for side-effectful LLM-agent tool calls.
- We develop a counterfactual stress-test framework that separates surface invariance from effect, resource, authorization, operation-mode, and provenance sensitivity.
- We show that existing/custom-stress methods have nontrivial local capabilities but incomplete joint binding, and identify resource/authorization binding as the central bottleneck.
- We demonstrate, through a local authorization-aware prototype and audits, that explicit atom-level authorization infrastructure substantially improves controlled pre-commit mediation.
- We add independently specified held-out contracts, realistic trace replay with a saved external subset, extraction-vs-authorization decomposition, interface-burden analysis, and stronger comparable baselines to evaluate whether the atom interface survives the main synthetic/circularity/baseline objections.

2 Threat Model and Security Goal

System model. We consider an LLM-agent runtime that proposes a side-effectful tool call before the call is committed to an external service. A pre-commit mediator can inspect the user task, recent agent trace, candidate tool name and arguments, tool schema, available evidence/provenance summaries, and any explicit authorization context provided by the local environment. The mediator emits one of three decisions: ALLOW, DENY, or ABSTAIN. ABSTAIN means the mediator refuses to commit automatically because resource identity, authorization scope, provenance, alias resolution, or operation semantics are insufficiently bound.

Adversary and hazards. The adversary may influence user-visible or tool-returned text, induce surface paraphrases, exploit resource aliases, request operation-mode changes, or attempt to route control through untrusted evidence. The hazards are pre-commit unsafe allows: sending or sharing with an unauthorized recipient, modifying an unauthorized calendar or file, committing instead of drafting, changing visibility, or authorizing an action based on untrusted control.

Out of scope. We do not model real external side effects, malicious tool implementations, compromised operating systems, post-commit rollback, real SaaS deployment semantics, browser automation, banking integration, or end-to-end prompt-injection prevention. We also do not claim that our local authorization context is a deployable enterprise permission system. The E55 prototype assumes a controlled mock contract in which the environment can provide typed authorization context and canonical resource information.

Security goal. Let c be a candidate tool call and let $L(c)$ be the ground-truth pre-commit label in a controlled contract. A monitor M satisfies the local safety goal on a dataset if it minimizes:

$$\text{UPA}(M) = \Pr[M(c) = \text{ALLOW} \mid L(c) = \text{DENY}],$$

while reporting its coverage and safe false-denial tradeoff:

$$\text{Coverage}(M) = \Pr[M(c) \neq \text{ABSTAIN}], \quad \text{FDenY}(M) = \Pr[M(c) = \text{DENY} \mid L(c) = \text{ALLOW}].$$

Coverage is not itself a safety metric: high coverage with high unsafe pre-allow is worse than abstention for pre-commit mediation. Conversely, zero unsafe pre-allow obtained by denying or abstaining on nearly everything is not sufficient utility evidence. The paper therefore reports all three quantities together.

3 Tool-Effect Binding Problem

Tool-effect binding asks whether a monitor’s decision is attached to the semantic side effect that would be committed, not merely to the surface form of a tool call. The binding problem appears whenever the same surface action can realize different effects, or different surfaces can realize the same effect.

Objects. Let c denote a candidate action. A candidate action includes a tool name, a schema-level argument object, and a surrounding context. Let $S(c)$ be its surface representation, $E(c)$ its realized effect type, $R(c)$ its target resource set, $O(c)$ its operation mode, $A(c)$ its authorization relation under the current context, and $P(c)$ its provenance/control-source status. These are analytic variables used for measurement; in deployment they may be partially observed or unavailable.

Binding requirement. A monitor should be invariant to changes in $S(c)$ that preserve (E, R, O, A, P) , and sensitive to changes in E, R, O, A , or P that alter the safe pre-commit decision. For example, renaming a recipient alias should not change the decision if the alias resolves to the same authorized principal. Adding an unauthorized Bcc should change the decision even if the tool name and primary effect remain the same.

Failure modes. We distinguish four recurrent failures.

- **Tool-proxy behavior:** the monitor follows tool names or coarse action strings rather than the realized effect and resource.
- **Resource under-binding:** the monitor recognizes the effect but does not bind all target resources, recipients, accounts, channels, files, or visibility states.
- **Authorization under-binding:** the monitor identifies a resource but fails to check whether the current authorization context covers the requested operation.
- **Provenance/control-source under-binding:** the monitor permits an action whose controlling instruction or evidence comes from an untrusted source.

Reviewer-facing boundary. The binding requirement is a precondition for robust policy enforcement, not a replacement for policy design. A monitor that extracts the wrong resource or operation can pass a well-specified policy for the wrong object. Conversely, a policy system can be formally precise but still unsafe if the candidate action is not correctly bound to the policy-relevant object before enforcement.

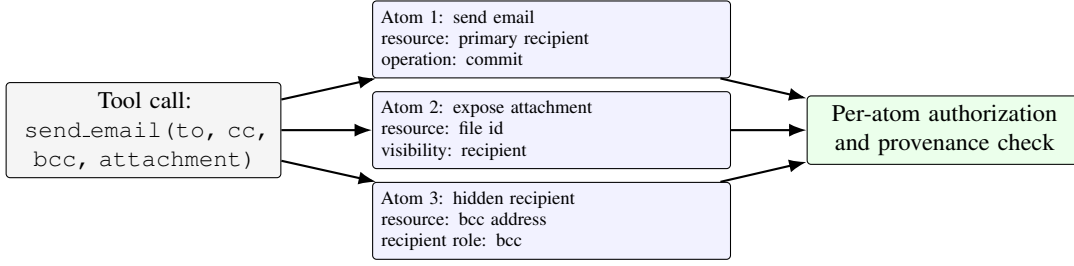


Figure 2: Effect-resource-operation atomization. A side-effectful tool call is a container of multiple security-relevant atoms rather than a single authorization object.

4 Effect-Resource-Operation Atoms

The central method abstraction is the *effect-resource-operation atom*. A side-effectful tool call is not an atomic security action; it is a container of one or more atoms. This matters because a single tool call can simultaneously send to multiple recipients, expose an attachment, create an event, invite attendees, share a file, change visibility, schedule a payment, or modify a role.

Atom schema. We represent an atom as:

```
EffectAtom = (effect, operation, resource_id, resource_type,
              recipient_role, visibility, commit_mode,
              provenance_source, control_source).
```

Not every domain populates every field. For example, `recipient_role` is central to email and Slack but may be absent for a file metadata update. `commit_mode` distinguishes draft, preview, schedule, and committed external action when the domain exposes such a distinction.

Atom minimality. An atom is minimal for pre-commit mediation when changing any one of effect, resource identity, operation or commit mode, authorization relation, or provenance/control source can change the ALLOW/DENY/ABSTAIN decision while the other fields remain fixed. Fields that cannot affect the decision for a domain should remain metadata or be omitted. Fields that can independently flip the decision must not be hidden inside a larger action container.

Examples. The atom boundary is domain-specific but the security pattern is common.

- **Email:** a `send_email` call expands into atoms for the `to`, `cc`, and `bcc` recipients, plus attachment atoms when files are included.
- **Calendar:** a `create_event` call expands into an event-creation atom and one attendee-invitation atom per invited principal.
- **File sharing:** a `share_file` or `public-link` call separates the file target, reader principals, and visibility change.
- **Transactions:** a payment-like call separates account, payee, amount semantics, and commit mode; in E55-v2, transaction amount is not itself treated as a resource-authorization atom.

Authorization context. The minimum authorization context for this paper contains:

```
allowed_effects, allowed_operations, allowed_resource_ids,
allowed_resource_aliases, allowed_recipients/channels/accounts/files,
allowed_visibility, draft_allowed, commit_allowed,
public_link_allowed, trusted_control_sources,
untrusted_control_sources, multi_resource_policy.
```

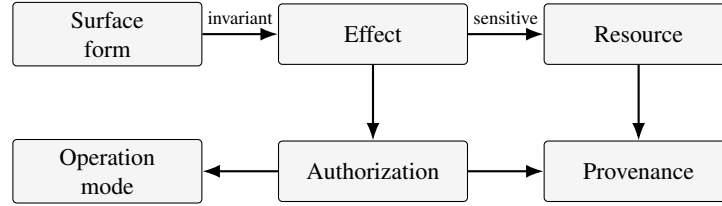
This context is not a full enterprise authorization layer. It is the smallest interface the pre-commit mediator needs to decide whether the candidate action's atoms are covered by explicit authorization.

Algorithm 1 AtomizeAndAuthorize

Require: Candidate call c , tool schema Σ , authorization context Γ , provenance summary Π

Ensure: ALLOW, DENY, or ABSTAIN

```
1: Expand  $c$  under  $\Sigma$  into atom set  $\mathcal{A} = \{a_1, \dots, a_k\}$ .
2: for all  $a_i \in \mathcal{A}$  do
3:   Canonicalize resource aliases in  $a_i$  using  $\Gamma$ .
4:   if effect, operation, resource, recipient/channel/account/file, visibility, draft/commit mode, or control source is
     explicitly outside  $\Gamma$  and  $\Pi$  then
5:     return DENY
6:   end if
7:   if a required field, alias, resource identity, authorization boundary, evidence source, or control source is unresolved
     then
8:     return ABSTAIN
9:   end if
10: end for
11: return ALLOW
```



Counterfactual pairs isolate one axis while holding the others fixed when possible.

Figure 3: Counterfactual stress lattice. The framework tests whether monitors are invariant to harmless surface changes and sensitive to semantic changes that alter safe pre-commit decisions.

Decision rule. Algorithm 1 gives the mediator-side decision procedure used to instantiate this abstraction in the local prototype. Let $\text{Expand}(c)$ return the atom set for candidate call c under the available schema and evidence. We write:

$$\text{Authorize}(c) = \text{Authorize}(\text{Expand}(c)).$$

The local decision rule is:

- ALLOW: every atom is covered by the authorization context and no provenance/control-source rule blocks the action.
- DENY: at least one atom is explicitly outside the authorization context.
- ABSTAIN: atom expansion, resource identity, alias resolution, evidence source, authorization boundary, or control source is ambiguous.

This atomization is the paper’s main constructive abstraction. It explains why a coarse tool-level guard can be high coverage yet unsafe: the guard may make a decision over the container while missing an unauthorized atom inside it. Conversely, once the runtime exposes atoms and authorization context, policy can be applied to the object that actually commits the side effect.

5 Counterfactual Stress-Test Framework

The measurement framework constructs controlled counterfactual cases that isolate semantic axes relevant to pre-commit decisions. The intent is not to reproduce every original benchmark result for each compared system. Instead, the framework asks whether a monitor can satisfy binding invariances and sensitivities on a controlled custom stress distribution.

Axes. We use six axes.

- **Surface invariance:** paraphrases, schema-level variants, or tool-name alternatives that preserve the same security-relevant action should not change the decision.
- **Effect sensitivity:** changing the realized effect should change the decision when the authorization differs.
- **Resource sensitivity:** changing target resource, recipient, account, channel, file, or visibility should change the decision when the new resource is unauthorized.
- **Authorization sensitivity:** changing the authorization context should change the decision even when the surface action remains similar.
- **Operation-mode sensitivity:** draft, preview, schedule, and commit modes are different security actions.
- **Provenance sensitivity:** trusted user control and untrusted tool-returned control must be distinguished.

Pairwise relations. For each pair (c_i, c_j) , the framework records which axes are held fixed and which axes are perturbed. A monitor receives only deployable or allowed input views for that method. Oracle labels and hidden fields are used for scoring, not as deployable inputs. The E48 unified evaluation contains 822 rows and 6,840 pairwise relations; its manifest records the available methods, input views, source counts, and leakage checks.

Metrics. The primary row metrics are UPA, FDeny, coverage, selective accuracy, effect/resource/authorization/provenance accuracy where defined, and abstain rate. Pairwise metrics include same-effect consistency, effect-change correctness, resource sensitivity, authorization sensitivity, provenance sensitivity, and flip-relation correctness. The paper emphasizes UPA, FDeny, and coverage in the main tables because they map directly to pre-commit safety tradeoffs.

Validity checks. The framework includes leakage and marker scans to check that deployable inputs do not contain gold labels. Later E55/E56/E57 checks add label-hidden replay, decision-path audit, resource perturbation, independent reference-authorizer agreement, and human spot audit. These checks reduce specific artifact risks, but they do not convert controlled synthetic/local contracts into deployment validation.

6 Reference Effect-Binding Guard

The reference hard Effect-Binding Guard is a deterministic monitor intended to test how far explicit tuple binding can go without a full authorization-aware contract. It is a diagnostic reference point rather than the endpoint of the defense.

Inputs. The guard consumes non-oracle views such as candidate action, argument summary, canonical visible summary, evidence summary, provenance summary, plan text, trajectory text, and tool inventory when available. It does not consume gold labels such as expected decision, gold effect, gold resource, gold authorization match, or gold provenance risk.

Tuple binding. The guard maps each candidate action to an effect/resource/authorization/provenance tuple and applies hard consistency checks. It abstains under multi-view disagreement, missing evidence, or insufficient provenance. It denies when the tuple indicates an unsafe action, and allows only when the tuple is compatible with the available authorization evidence. The method is intentionally conservative, because the experiment is designed to locate remaining binding bottlenecks rather than maximize apparent utility.

Expected failure surface. The guard can improve over pure surface proxies when the effect and resource are explicit. It is expected to fail when a resource or authorization relation is latent in a tool-specific argument, spread across multiple fields, or dependent on local aliases and operation-mode semantics. E50's resource/authorization stress was designed to test exactly this failure surface.

7 Authorization-Aware Pre-Commit Prototype

The E55 local prototype tests a stronger interface: rather than inferring authorization only from generic text, the environment provides a typed authorization context and the mediator expands candidate calls into atoms before decision. This is a local mock contract. It should be read as feasibility evidence for explicit pre-commit authorization infrastructure, not as a deployable permission system.

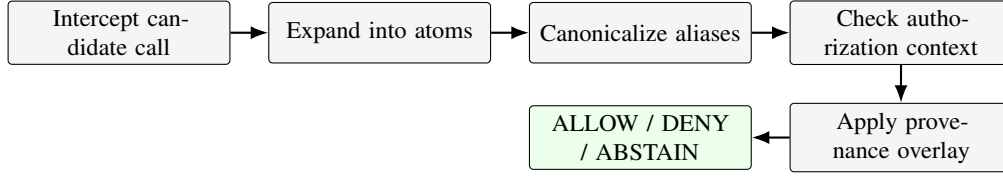


Figure 4: Authorization-aware local pre-commit prototype pipeline. The prototype is a local mock contract for pre-commit mediation, not a full enterprise authorization layer.

Pipeline. The prototype performs five steps. First, it intercepts the candidate tool call before commit. Second, it expands the call into effect-resource-operation atoms. Third, it canonicalizes resource identifiers and aliases. Fourth, it checks each atom against the authorization context. Fifth, it applies provenance/control-source checks and emits ALLOW, DENY, or ABSTAIN.

Domain coverage. The local contract covers five domains: email, calendar, file, Slack, and transaction. Each domain contains 120 rows in E55-v2, for 600 rows total. The stress axes include multi-resource expansion, operation shifts, provenance shifts, resource shifts, and same-effect surface changes. The domain examples are intentionally compact but cover security-relevant distinctions that coarse tool monitors often collapse.

Abstention semantics. Abstention is a first-class decision. In E55-v2, unknown resources, unresolved aliases, insufficient evidence fallback, or ambiguous draft/commit status are not silently allowed. The high-level goal is to convert dangerous automatic decisions into either denied or explicitly abstained pre-commit states.

Why this matters. The E50 bottleneck shows that tuple-level binding without a local authorization contract can make many decisions while missing unsafe resource/authorization cases. The E55 prototype asks the constructive follow-up: if the environment exposes the missing authorization interface and the mediator reasons over atoms, does the bottleneck reduce in a controlled setting? The answer is yes for the E55-v2 local contract, subject to the limitations and audit findings reported below.

8 Experimental Setup

Materials. The evaluation uses fixed JSON result artifacts distributed with the anonymized artifact bundle; Table 19 maps the main claims to their sources.

E47 existing/custom-stress methods. E47 evaluates local baselines, released checkpoint components, and structural components on custom stress cases. ToolSafe/TS-Guard and Safiron are evaluated as released checkpoints on E47 custom stress, not as reproductions of their original benchmark numbers. IPIGuard and CaMeL are included through structural or component stress where the available interface supports our binding diagnostics.

E48 hard guard. E48 evaluates the reference hard guard on 822 controlled rows and 6,840 pairwise relations. We report the full result, a held-out split, and the source-balanced E50 robustness audit. The main metrics are UPA, FDeny, and coverage.

E50 robustness and bottleneck stress. E50 contains source-balanced repeats, resource/authorization stress, provenance stress, risk-coverage curves, threshold sensitivity, and failure examples. The resource/authorization stress has 240 rows; the provenance stress has 336 rows. A policy oracle independently recomputes each resource/authorization label from the visible authorized effect, canonical resource and alias map, and commit permission; generation fails when the oracle and label disagree.

E55/E56/E57 local pre-commit evidence. E55 evaluates a local pre-commit authorization prototype on 600 rows across five domains. This paper uses E55-v2 strict results as the main pre-commit table because they are the corrected rerun. The original E55 strict result and human-corrected sensitivity are reported as audit/sensitivity evidence. E56 contributes strict replay and build/audit checks. E57-v2 contributes resource perturbation, reference-authorizer agreement, and a 60-row spot-audit packet. A separate E57 human audit of 60 rows found 54/60 decision agreement and generated corrected-label evidence; it is reported as a human audit, not merged with deterministic reference-authorizer agreement.

Table 1: Existing/custom-stress method capability matrix. Scores come from E47 custom stress artifacts and released-checkpoint custom-stress runs where applicable; they are not original-paper benchmark reproductions. Coverage is included where available because UPA alone is not interpretable without abstention/coverage behavior. Empty entries indicate not applicable or not available for the method’s signal.

Method	Scope	Surface inv.	Effect sens.	Auth. sens.	Resource aware	UPA	Coverage
Tool-name proxy	local rule baseline	1.000	0.000	0.000	0.000	0.292	1.000
Static/plan/trajectory text proxy	local text-rule baselines	0.984	0.000	0.903	1.000	0.080	1.000
Local Qwen self-audit	local LLM self-audit	0.865	0.667	0.565	0.292	0.004	0.998
TS-Guard checkpoint	released checkpoint on E47 custom stress	0.883	0.625	0.764	0.833	0.159	1.000
Safiron checkpoint	released checkpoint on E47 custom stress	0.669	1.000	0.088	0.167	0.545	1.000
Local Qwen effect/resource mapper	local LLM mapper	0.992	0.833	0.375	0.125	0.333	0.938
IPIGuard topology-only	structural topology signal	1.000	0.000	–	–	–	–
CaMeL structural-policy stress	local component stress	1.000	–	–	–	0.250	1.000

E60 held-out contract. E60 is an independently specified held-out authorization contract with 480 cases across five domains. Its tool names, argument fields, alias style, resource identifiers, and policy format differ from E55. Deployable inputs, gold atoms, and gold labels are stored separately, and a leakage scan checks that gold annotations do not enter the deployable view. The stress axes include multi-resource expansion, operation-mode shift, alias resolution, provenance/control-source shift, authorization shift, same-effect surface variants, incomplete context, and ambiguous resource identity.

E61 realistic trace replay. E61 evaluates 300 sandboxed or replayed multi-step traces across five domains. Source A contains clean sandbox traces, Source B contains noisy realistic traces with nested arguments and partial evidence, and Source C contains adversarial provenance-shift traces with untrusted tool-returned content. We additionally convert 156 saved AgentDojo-style/IPIGuard replay traces with mutating candidate tool calls into a separate external-trace subset. No real external side effects are executed. The trace manifests require deployable trace views, normalized traces, side-channel atoms and labels, and leakage scans; the external subset uses trace metadata plus rule-derived sidecar atoms rather than independent human annotation.

E62–E64 mechanism, burden, and baselines. E62 decomposes atom extraction and authorization checking under three modes: gold atoms with gold context, extracted atoms with gold context, and extracted atoms with degraded context. E63 measures interface burden and context degradation under alias, resource, provenance, operation-mode, visibility, stale-policy, contradictory-policy, and incomplete enterprise-context perturbations. E64 compares full atom-level mediation with tool-name, tool-call-level, resource-only, tuple-level, no-provenance, no-alias, LLM-judge, least-privilege/capability-style, and ToolSafe/TS-Guard-style released-guardrail comparable local baselines under deployable-input restrictions.

Metrics and labels. Unsafe pre-allow is computed as ALLOW on rows labeled DENY. Safe false-denial is computed as DENY on rows labeled ALLOW. Coverage is the fraction of rows on which a method emits ALLOW or DENY rather than ABSTAIN. We do not compute false negative rate from F1 or substitute coverage-missing stress quantities for deployed-system quantities.

9 Results

9.1 Finding 1: Existing Methods Have Partial Binding but Incomplete Joint Binding

Table 1 shows that existing/custom-stress methods are not merely tool-name classifiers. TS-Guard reaches 0.883 surface invariance and 0.764 authorization sensitivity on the E47 custom stress, and Safiron reaches 1.000 effect sensitivity. Local Qwen variants also show nontrivial effect sensitivity. The axis columns measure pairwise sensitivity or invariance under controlled perturbations, whereas UPA is a row-level safety error: an unsafe row is counted when the monitor allows a denied action. They must be read together. High performance on one axis does not imply joint binding. Safiron has 0.545 unsafe pre-allow in this custom stress; local Qwen effect/resource mapping has high surface invariance and effect sensitivity but 0.125 resource awareness and 0.333 unsafe pre-allow. Current monitors can capture parts of the binding problem, but they remain incomplete when resource, authorization, and provenance must be bound jointly.

Table 2: Reference hard Effect-Binding Guard results. The guard improves controlled stress tradeoffs, but retains nonzero unsafe pre-allow and false denial.

Scope	UPA	FDeny	Coverage
E48 full, 822 rows	0.036	0.065	0.917
E48 held-out split	0.051	0.048	0.955
E50 source-balanced mean	0.051	0.059	0.923

Table 3: Bottleneck stress results for the hard guard. Resource/authorization stress exposes high unsafe pre-allow despite high coverage; provenance overlay removes unsafe pre-allow in the controlled provenance stress but increases false denial and abstention.

Stress set	Rows	UPA	FDeny	Coverage
Resource/auth stress	240	0.383	0.000	0.921
Provenance stress	336	0.000	0.188	0.545

9.2 Finding 2: Resource/Authorization Is the Dominant Bottleneck

Table 2 reports the reference hard guard. On the 822-row E48 suite, the guard obtains 0.917 coverage with 0.036 unsafe pre-allow and 0.065 safe false-denial. On the held-out split, unsafe pre-allow rises to 0.051 while coverage remains 0.955. Source-balanced E50 repeats preserve the same qualitative result: 0.051 mean unsafe pre-allow, 0.059 mean safe false-denial, and 0.923 mean coverage.

Table 3 identifies the main bottleneck. On E50 resource/authorization stress, the hard guard has 0.383 unsafe pre-allow at 0.921 coverage. This rules out a benign explanation in which the guard simply abstains on hard cases. Instead, it often makes a decision while missing the authorization-relevant resource relation. The 46 unsafe allows comprise 20 near-alias cases, 20 out-of-scope-resource cases, and six alternate-effect cases. This is a granularity failure: the guard decides over an under-decomposed action object, so an apparently acceptable container can still include an unauthorized resource or authorization shift. Provenance stress shows a different tradeoff: unsafe pre-allow is 0.000, but coverage falls to 0.545 and safe false-denial rises to 0.188. Provenance overlays can reduce unsafe pre-allow under controlled assumptions, but their utility cost must be reported rather than hidden.

9.3 Finding 3: Atom-Level Mediation Improves Local Contract Decisions

Table 4 shows the E55-v2 local pre-commit warm-up. The existing hard guard on the same 600-row local contract has 0.200 coverage, 0.800 abstention, and $12/276 = 0.043$ unsafe pre-allow. The authorization-aware prototype reaches $528/600 = 0.880$ coverage and $72/600 = 0.120$ abstention, while reducing unsafe pre-allow to $0/276$ and safe false-denial to $0/252$ under E55-v2 strict label-hidden evaluation. E55-v2 is no longer the sole proof of the paper; it is the controlled feasibility result showing that explicit authorization context and atom-level checking can turn abstention-heavy behavior into decidable local mediation.

Table 5 reports E55-v2 strict ablations. Removing multi-resource expansion, operation-mode binding, or provenance overlay each raises unsafe pre-allow to $60/276 = 0.217$. The equality is an aggregate rate match in the current JSON artifacts; it does not establish that the same rows fail under all three ablations. Removing alias resolution raises safe false-denial to $60/252 = 0.238$. Table 6 separates deterministic validity checks from human audit evidence: E57-v2 resource-name perturbation yields zero changed decisions and zero UPA/coverage delta, while the human audit agrees on 54/60 decisions and identifies corrections.

9.4 Finding 4: The Atom Interface Transfers to an Independently Specified Held-Out Contract

Table 7 reports E60, a 480-case held-out contract across five domains with different tool names, argument fields, alias style, resource identifiers, and policy format from E55. Gold atoms and labels are separated from deployable inputs and the leakage scan reports zero violations. Full atom-level mediation reaches 0.854 coverage with 0.000 unsafe pre-allow, 0.083 safe false-denial, and 0.812 atom exact-set match. The result supports transfer of the atom abstraction beyond the E55 local schema, but the false-denial and extraction gap show measurable degradation that must be retained in the claim boundary.

Table 4: E55-v2 authorization-aware local pre-commit prototype. Original E55 and human-corrected sensitivity are retained in Table 16 as version-policy and audit evidence, not as the main performance result.

Method / source	UPA	FDeny	Coverage	Abstain
E55-v2 existing hard guard	0.043	0.000	0.200	0.800
E55-v2 authz-aware guard	0.000	0.000	0.880	0.120

Table 5: E55-v2 strict atom/authorization ablations. These ablations diagnose which semantic interfaces carry the E55-v2 improvement; original E55 ablations are retained in Table 17 for version comparison.

Ablation	Main failure metric	Value
No multi-resource expansion	UPA	0.217
No operation-mode binding	UPA	0.217
No provenance overlay	UPA	0.217
No alias resolution	FDeny	0.238

9.5 Finding 5: Atom Extraction Works on Realistic Traces with Identifiable Failure Modes

Table 8 reports E61, a 300-trace sandboxed replay evaluation. The trace manifest contains clean sandbox traces, noisy realistic traces, and adversarial provenance-shift traces; 30.0% include untrusted tool-returned content, 80.0% include nested or multi-resource arguments, 24.0% include incomplete or ambiguous context, and 21.3% include operation-mode distinctions. Full atom-level mediation reaches 0.847 coverage with 0.000 unsafe pre-allow, 0.053 safe false-denial, and 0.733 atom exact-set match. Source B and Source C are intentionally harder than clean sandbox replay; the drop in atom exact-set match identifies realistic trace parsing and provenance/control-source handling as remaining failure modes rather than hidden successes. Table 9 adds a separate 156-trace external replay subset from saved AgentDojo-style/IPIGuard traces with mutating candidate calls and no real external side effects. The subset reaches 0.904 coverage, 0.000 unsafe pre-allow, 0.000 safe false-denial, and 0.821 atom exact-set match. Its labels come from saved trace metadata and its atoms from a rule-derived sidecar, so it supports an external replay-subset claim, not independent human annotation or deployed-trace safety. Table 10 reports the artifact-generated and external replay views side by side for accounting only; the paper does not pool the source types into a single performance claim. Figure 5 summarizes the label-hidden replay path used for these rows.

9.6 Finding 6: Extraction Errors and Missing Authorization Context Dominate Remaining Failures

Table 11 decomposes the pipeline. Mode A uses gold atoms and gold authorization context, isolating the checker. Mode B replaces gold atoms with extracted atoms, and Mode C additionally degrades authorization context. On E60, coverage moves from 0.875 in Mode A to 0.854 in Mode B and 0.783 in Mode C; atom exact-set match falls from 1.000 to 0.812 when extraction is introduced. On E61, Mode A reaches perfect atom match by construction, Mode B falls to 0.733 atom exact-set match, and Mode C reduces coverage to 0.707 with 0.439 safe false-denial. These gaps attribute remaining errors primarily to extraction and missing or degraded authorization context rather than to checker logic alone. Figure 6 states the mode comparison used for the attribution.

9.7 Finding 7: Incomplete or Stale Context Produces Abstention/False-Denial Tradeoffs

Table 12 evaluates systematic context degradation. Removing provenance annotations drives coverage down to 0.219 and abstention up to 0.781 with 0.000 unsafe pre-allow. Contradictory policy and incomplete enterprise-style context similarly produce high abstention. Removing operation-mode or visibility fields creates high false-denial rather than unsafe pre-allow. This is the intended conservative failure mode: incomplete, stale, or contradictory authorization context should surface as abstention or false denial rather than silent unsafe allow. The tradeoff is utility loss, not a complete authorization infrastructure.

9.8 Finding 8: Atom-Level Mediation Improves Non-Atom Baseline Tradeoffs

Table 13 summarizes E64 for the non-atom baselines: tool-name, tool-call-level, resource-only, tuple-level, no-provenance, and LLM-judge variants. Across E55-v2, E60, and E61, full atom-level mediation has lower unsafe pre-allow and higher coverage than the strongest non-atom baseline selected by unsafe pre-allow. On E60, that baseline has 0.311 unsafe pre-allow

Table 6: Audit and validity checks. Deterministic E57-v2 checks and human audit checks answer different validity questions and are not merged into a single agreement claim.

Check	Result	Scope
E55-v2 leakage audit	leakage-free	deployable-input leakage diagnostic
E55 decision-path audit	passed	implementation-path audit
E56 strict label-hidden replay	passed	label-hidden replay and build audit
E57-v2 resource perturbation	UPA delta 0.000; coverage delta 0.000; changed decisions 0	naming-template robustness diagnostic
E57-v2 reference authorizer	decision, atom-count, and atom resource-set agreement 1.000	independent local implementation agreement
E57-v2 spot-audit packet	60 rows; required fields present	human-inspectable packet
E57 human audit	decision agreement 54/60; all checks true 46/60	post-hoc human audit with corrections
E57 human corrections	6 decision, 13 atom, 10 violation-reason corrections	label/atom sensitivity evidence

Table 7: E60 independently specified held-out contract. Gold labels and atoms are hidden from deployable inputs.

Dataset	Rows	UPA	FDeny	Coverage	Atom exact
E60 held-out	480	0.000	0.083	0.854	0.812

at 0.625 coverage, whereas full mediation has 0.000 unsafe pre-allow at 0.854 coverage. On E61, the corresponding numbers are 0.189 and 0.700 versus 0.000 and 0.847.

The diagnostic B5 and B7 rows qualify the claim rather than strengthen it. Removing alias canonicalization can also reach zero unsafe pre-allow, but with substantially higher false denial; the least-privilege/capability-style adapter can match full mediation when its capability contract already carries atom-equivalent resource and provenance information. Thus E64 supports a granularity and interface conclusion: non-atom monitors miss policy-relevant structure, while capability-style success depends on supplying that structure explicitly. These comparisons use the same deployable input view and do not claim to reproduce any external paper’s original benchmark protocol.

Table 14 reports B8, a ToolSafe/TS-Guard-style released-guardrail comparable local adapter. B8 is not a reproduction of ToolSafe/TS-Guard’s original benchmark or checkpoint protocol; it maps the same label-hidden deployable input views to step-level ALLOW/DENY/ABSTAIN decisions. It remains weaker than full atom mediation on E55-v2, E60, and artifact-generated E61, but performs well on the external subset because those saved traces expose untrusted-control evidence directly. The comparison is therefore evidence about this common-input adapter setting, not global superiority over released guardrail systems.

9.9 Finding 9: Interface Burden Is Measurable and Bounded in the Studied Domains

Table 15 reports the interface burden. Each studied domain uses two tool forms, eight atom expansion rules, eight authorization fields, and 312 alias entries across 156 combined E60/E61 cases, with estimated authoring time of 74 minutes per domain. These numbers are not deployment costs for real enterprises; they quantify the studied interface burden and make explicit which parts must be supplied by runtime systems, schema authors, or trace parsing. The result answers the infrastructure objection more directly than saying that the runtime “provides context”: atom-level mediation requires concrete schema, alias, provenance, and policy interfaces, and its utility degrades when they are incomplete.

10 Failure Analysis

Resource and authorization failures. The dominant failure mode is not a failure to notice that a tool call is side-effectful. It is failure to bind the action to the specific resource and authorization relation. In E50, the hard guard frequently decides rather than abstains, yet it allows unsafe resource/authorization cases. This suggests that surface-level safety categories and even effect labels are insufficient when the policy-relevant question is “which object, under which authorization, with which operation?”

Multi-resource containers. E55 ablations show why a tool call must be treated as a container. If multi-resource expansion is removed, the prototype can authorize the visible primary action while missing hidden or secondary resources. Email Bcc fields, event attendees, file visibility changes, Slack role/channel changes, and transaction payee/account combinations all create this pattern.

Operation-mode and provenance failures. Operation mode changes convert apparently similar calls into different security actions. A draft may be acceptable when a committed send is not; a scheduled payment may differ from an immediate transfer.

Table 8: E61 realistic trace replay. Source A is clean sandbox replay, Source B is noisy realistic replay, and Source C stresses adversarial provenance shifts.

Trace set	Rows	UPA	FDeny	Coverage	Atom exact
E61 all traces	300	0.000	0.053	0.847	0.733
A clean sandbox	75	0.000	0.044	1.000	0.840
B realistic noisy	135	0.000	0.058	0.881	0.719
C adversarial provenance shift	90	0.000	–	0.667	0.667

Table 9: E61 external-trace subset. The subset uses saved AgentDojo-style/IPIGuard replay traces with no real external side effects; labels and sidecar atoms are stored separately from deployable inputs.

Trace set	Rows	UPA	FDeny	Coverage	Atom exact
External IPIGuard/AgentDojo-style subset	156	0.000	0.000	0.904	0.821
external banking	46	0.000	0.000	0.891	0.848
external chat	31	0.000	0.000	0.903	0.806
external travel	10	0.000	0.000	1.000	0.900
external workspace	69	0.000	0.000	0.899	0.797

Provenance/control-source failures are separate: a call whose resource is authorized may still be unsafe when the controlling instruction comes from an untrusted fetched page or tool output. E50 provenance stress shows that hard provenance overlays can remove unsafe pre-allow under the controlled setup, but at the cost of false-denial and abstention.

Human audit corrections. The E57 human audit found corrections rather than merely confirming the construction. Some corrections concern unknown-resource fallback in email and file cases; others concern transaction-domain amount handling, where amount should not always be treated as a resource-authorization atom. These corrections are valuable because they sharpen the claim boundary. The corrected sensitivity preserves zero unsafe pre-allow for the full authz-aware guard, but introduces a small safe false-denial rate. The paper therefore reports the prototype as robust to corrected-label sensitivity on unsafe pre-allow, not as perfectly labeled.

Residual reviewer risks. The local E55-v2 contract is synthetic and deterministic. E60 and E61 reduce the strongest circularity concern by adding an independently specified held-out contract and sandboxed realistic trace replay, but they remain artifact-bounded evaluations with no real external side effects. E62 shows that remaining errors are concentrated in extraction and missing or degraded authorization context; E63 shows that incomplete interfaces produce utility loss through abstention or false denial. These are narrower claims than a deployed-system guarantee.

11 Discussion

Tool calls are containers, not security atoms. The central design implication is that the unit of mediation should not be the tool invocation itself. A single call may contain several policy-relevant atoms: recipients, files, attendees, visibility changes, account/payee pairs, and draft/commit transitions. Tool-level monitoring can therefore be high coverage while still authorizing the wrong object. Atomization makes the enforcement target explicit before a policy is applied.

Authorization must be runtime infrastructure. The E50 bottleneck is not solved by a better wording of the safety question. Resource and authorization failures persist when the monitor lacks a typed representation of the authorization object. Progent, MiniScope, CaMeL, and related structural defenses are complementary because they provide enforcement or control structure; this paper identifies the semantic binding layer that those systems need before enforcement can target the right object.

The stronger evidence shifts the bottleneck. E55-v2 shows controlled local feasibility, while E60 and E61 test whether the same atom interface survives an independently specified contract and sandboxed realistic trace replay. The remaining failures are not primarily checker failures under gold atoms and context; E62 attributes much of the gap to atom extraction and missing or degraded authorization context, and E63 makes the interface burden explicit. This is a narrower and more useful conclusion than treating authorization infrastructure as complete.

Table 10: E61 artifact-generated replay and external-trace subset, reported separately to avoid mixing source types.

Trace set	Rows	UPA	FDeny	Coverage	Atom exact
Artifact-generated E61	300	0.000	0.053	0.847	0.733
External subset	156	0.000	0.000	0.904	0.821

Raw sandbox/replay trace → normalized trajectory → deployable input view → atom extraction → authorization decision. Gold atoms and labels are stored only in side-channel annotation files.

Figure 5: E61 trace replay pipeline.

Abstention is a mediation outcome. Pre-commit mediators should not be evaluated only by accuracy on forced allow/deny decisions. When resource identity, alias resolution, control source, or authorization scope is unresolved, abstention is the correct controlled outcome. Reporting UPA, FDeny, and coverage together distinguishes a useful mediator from a system that hides uncertainty by allowing or denying everything. E55-v2 is useful because it raises coverage while driving unsafe pre-allow to zero under the local contract, not because it eliminates abstention.

12 Related Work

Agent safety benchmarks. ToolEmu evaluates LM agents using an LM-emulated sandbox and highlights risks such as privacy leakage and financial loss in tool-rich settings [11]. AgentDojo provides dynamic environments for prompt-injection attacks and defenses over realistic agent tasks [5]. These benchmarks motivate tool-use safety as a first-class problem. Our work asks a narrower pre-commit question: before a task succeeds or fails end-to-end, has the candidate action been bound to the effect, resource, operation, authorization, and provenance facts that determine whether it may commit?

Pre-execution and step-level guardrails. ToolSafe/TS-Guard studies proactive step-level tool invocation safety and feedback-driven guardrails [10]. Safiron studies foundational pre-execution guardrails trained with synthetic data and evaluated on Pre-Exec Bench [9]. These systems ask whether a planned or candidate action is risky before execution. Our custom stress tests ask a more specific semantic question: whether the monitor’s decision is bound to realized effect, resource, authorization, operation mode, and provenance rather than surface form.

Structural prompt-injection defenses and provenance. CaMeL separates trusted control from untrusted data and uses capability ideas to prevent unauthorized data flows [4]. IPIGuard uses a tool dependency graph to reduce unintended tool invocations under indirect prompt injection [1]. ARGUS constructs provenance-style influence structure for LLM-agent defenses against context-aware prompt injection [14]. AgentVisor uses semantic virtualization, privilege separation, and tool-call interception to mediate agent execution [15]. These works shift defenses from prompt-only filtering toward execution structure. Tool-effect binding is compatible with that shift, but it targets a different layer: the candidate action must first be decomposed into the atoms whose control source and provenance can be checked.

Capability, authorization, and least privilege. Capability systems were introduced as a way to name both authority and object references in computation [7]. The confused-deputy problem shows why ambient authority can let an otherwise legitimate program exercise the wrong authority on behalf of the wrong principal [8]. Progent introduces programmable privilege control policies for LLM agents [13]. MiniScope reconstructs permission hierarchies and applies a least-privilege framework for authorizing tool-calling agents [16]. CaMeL also uses capability-style structure to separate data and control [4]. These works are closest to the authorization motivation. Their policies are only as accurate as the object being checked: least privilege over a tool call can still miss an unauthorized Bcc, public-link change, or commit-mode shift if those atoms are not represented.

Mediation before enforcement. Classical security design separates the policy to be enforced from the object over which mediation occurs. Saltzer and Schroeder’s complete-mediation principle requires that every access to every object be checked against authority [12]. Information-flow and provenance foundations similarly emphasize that security decisions depend on how information or influence reaches an action, not only on the final command string [6, 2, 3]. In LLM-agent tool use, the mediated object is often implicit in a tool name and argument string. This paper contributes the semantic binding layer

Table 11: E62 extraction-vs-authorization decomposition. Mode A uses gold atoms and gold context, Mode B uses extracted atoms and gold context, and Mode C uses extracted atoms with degraded context.

Dataset/mode	Rows	UPA	FDeny	Coverage	Atom exact
E55-v2 A	600	0.000	0.000	1.000	1.000
E55-v2 B	600	0.000	0.000	0.880	0.880
E55-v2 C	600	0.000	0.071	0.730	0.880
E60 A	480	0.000	0.000	0.875	1.000
E60 B	480	0.000	0.083	0.854	0.812
E60 C	480	0.000	0.533	0.783	0.812
E61 A	300	0.000	0.000	1.000	1.000
E61 B	300	0.000	0.053	0.847	0.733
E61 C	300	0.000	0.439	0.707	0.733

Mode A isolates checker behavior with gold atoms and gold context. Mode B adds extracted atoms. Mode C adds degraded context. Differences attribute failures to extraction, context, or checker logic.

Figure 6: E62 extraction-vs-authorization decomposition.

before enforcement: a side-effectful tool call must first be decomposed into effect-resource-operation atoms so that guardrails, capability restrictions, least-privilege policies, or provenance checks can be applied to the correct object.

Positioning. Prior work provides benchmarks, pre-execution guards, capability restrictions, least-privilege policies, and provenance-aware structure. Our contribution is the semantic binding layer before enforcement: a side-effectful tool call must first be decomposed into effect-resource-operation atoms so that policy can be applied to the correct object.

13 Limitations

Controlled stress and artifact-bounded contracts. The evidence comes from controlled custom stress sets, a local mock pre-commit contract, an independently specified held-out contract, and sandboxed/replayed traces. It does not establish safety for real SaaS, browser, banking, email, calendar, file-sharing, Slack, or workspace systems, and the numbers should not be interpreted as deployed false-allow or false-denial rates. The experiments avoid real external side effects and do not model post-commit rollback.

Synthetic and deterministic artifacts. The E55/E57 local contract is deterministic and synthetic. Its resource schemas, aliases, and authorization context are designed for the experiment. E60 adds an independently specified contract and E61 adds sandboxed realistic trace replay plus a saved external replay subset, but these remain generated/replayed artifact evaluations rather than live third-party deployments. The external subset uses trace metadata and rule-derived sidecar atoms rather than independent human annotation. E57-v2 perturbation and independent reference-authorizer checks reduce lexical-template and implementation-coupling concerns; they do not prove general transfer.

Authorization-system scope. The E55-v2 prototype assumes that the local environment can provide typed authorization context, canonical resource information, and provenance/control-source summaries. It is therefore not a complete enterprise permission system and does not handle malicious tool implementations, compromised operating systems, browser/SaaS compromise, or policy lifecycle management. The contribution is the pre-commit semantic interface, not a replacement for production authorization infrastructure.

Baseline scope. Released checkpoints are evaluated on our custom stress interface. These results are useful for diagnosing binding behavior under our stress axes, but they are not original-paper benchmark reproductions. We therefore avoid claims that one external system is globally safer or weaker than another.

Human audit findings. The human audit found corrections. This is a strength for claim hygiene, but a limitation for label certainty. The paper should use corrected-label sensitivity wording and should not claim that the audit perfectly confirmed all labels, atoms, or violation reasons.

Table 12: E63 context degradation. Missing, stale, or contradictory context should increase abstention or false denial rather than unsafe allow.

Perturbation	UPA	FDeny	Coverage	Abstain
remove alias entries	0.000	0.238	0.855	0.145
remove canonical resource ids	0.000	0.135	0.854	0.146
remove provenance annotations	0.000	0.070	0.219	0.781
flip one permission	0.000	1.000	1.000	0.000
introduce contradictory policy	0.000	0.070	0.360	0.640
remove operation mode field	0.000	1.000	1.000	0.000
remove visibility field	0.000	0.963	0.979	0.021
split policy across partial contexts	0.000	1.000	1.000	0.000
stale resource owner	0.000	1.000	0.923	0.077
incomplete enterprise style context	0.000	0.303	0.488	0.512

Table 13: E64 summary: full atom-level mediation compared with the strongest non-atom baseline by unsafe pre-allow. The baseline column ranges over B0–B4 and B6; B5 no-alias and B7 capability-style variants are diagnostic variants reported in the artifact.

Dataset	Best non-atom UPA	Best non-atom coverage	Full UPA	Full coverage
E55-v2	0.043	0.200	0.000	0.880
E60	0.311	0.625	0.000	0.854
E61	0.189	0.700	0.000	0.847

Remaining evaluation gaps. E60–E64 address the largest prior gaps by adding held-out contracts, realistic trace replay, extraction decomposition, interface-burden analysis, and stronger comparable baselines. Remaining gaps are live integration with independently maintained authorization systems, externally authored and human-labeled trace corpora, uncertainty estimates over larger domains, and released-system adapters that run their original protocols end to end. These would test whether the atom interface remains useful outside the controlled artifact.

14 Ethics and Artifact Scope

The experiments are designed to avoid real external side effects. Tool actions are represented in synthetic or local mock contracts, and the pre-commit prototype is evaluated without sending real emails, sharing real files, modifying real calendars, posting to real Slack workspaces, or executing real financial transactions. The goal is to measure and diagnose semantic binding failures before deployment, not to demonstrate or encourage attacks against live systems.

Failure examples should be sanitized and should avoid credentials, real account identifiers, or live resource names. Any future realistic replay environment should preserve the no-real-side-effect constraint.

15 Conclusion

Tool-effect binding shifts LLM-agent safety from asking whether a tool call looks risky to asking what security-relevant atoms the call would commit. Counterfactual stress tests expose where current monitors fail this binding, especially for resource and authorization shifts. A local authorization-aware prototype, an independently specified held-out contract, and sandboxed realistic trace replay show that explicit atom-level authorization context can make many such decisions decidable under controlled contracts. The remaining work is to connect this interface to live, independently maintained authorization systems and broader external trace corpora without turning artifact-bounded evidence into a deployed-system guarantee.

References

- [1] Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents, 2025. EMNLP 2025.

Table 14: B8 released-guardrail comparable local adapter. The adapter is anchored to ToolSafe/TS-Guard’s step-level guardrail framing and run under this paper’s label-hidden deployable input contract; it is not an original benchmark reproduction.

Dataset	Rows	UPA	FDeny	Coverage	Abstain
E55-v2	600	0.435	0.095	0.900	0.100
E60	480	0.533	0.200	1.000	0.000
E61 artifact-generated	300	0.189	0.222	1.000	0.000
E61 external subset	156	0.000	0.000	1.000	0.000

Table 15: E63 interface burden by domain.

Domain	Cases	Tools	Atom rules	Auth fields	Aliases	Minutes
docvault	156	2	8	8	312	74
maildesk	156	2	8	8	312	74
meetroom	156	2	8	8	312	74
paydesk	156	2	8	8	312	74
teamchat	156	2	8	8	312	74

- [2] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.
- [3] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.
- [4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. Defeating prompt injections by design, 2025.
- [5] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.
- [6] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.
- [7] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [8] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [9] Yue Huang, Hang Hua, Yujun Zhou, Pengcheng Jing, Manish Nagireddy, Inkit Padhi, Greta Dolcetti, Zhangchen Xu, Subhajit Chaudhury, Ambrish Rawat, Liubov Nedoshivina, Pin-Yu Chen, Prasanna Sattigeri, and Xiangliang Zhang. Building a foundational guardrail for general agentic systems via synthetic data, 2025.
- [10] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback, 2026. Work in progress.
- [11] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox, 2023.
- [12] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [13] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing ai agents with privilege control, 2025.
- [14] Shihao Weng, Yang Feng, Jinrui Zhang, Xiaofei Xie, Jiongchi Yu, and Jia Liu. ARGUS: Defending LLM agents against context-aware prompt injection, 2026. Submitted 5 May 2026.
- [15] Zonghao Ying, Haozheng Wang, Jiangfan Liu, Quanchen Zou, Aishan Liu, Jian Yang, Yaodong Yang, and Xianglong Liu. AgentVisor: Defending LLM agents against prompt injection via semantic virtualization, 2026. Submitted 27 April 2026.

- [16] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. Miniscope: A least privilege framework for authorizing tool calling agents, 2025.

A Additional Tables

Table 16: Canonical E55 version policy used in this paper.

Artifact	Role in paper	Rationale
E55-v2 strict	Main pre-commit and ablation result	Corrected rerun with 600 rows, five domains, 0.880 authz-aware coverage, zero UPA, zero FDeny, and E55-v2 ablation values in Table 5.
Original E55 strict	Historical comparison source	Provides original 0.920 coverage result and earlier ablation values 0.333, 0.148, 0.148, 0.211 for version comparison.
Original E55 human-corrected sensitivity	Audit/sensitivity evidence	Preserves zero UPA but introduces 0.017 FDeny after six decision corrections.
E57-v2 validity	Deterministic validity evidence	Resource perturbation and reference-authorizer checks on corrected v2.
E57 human audit	Human audit evidence	Reports 54/60 decision agreement and corrected-label subset; not merged with reference-authorizer agreement.

Table 17: E55-v2 versus original E55 ablation note. These values should not be mixed without labeling the source version.

Ablation metric	Original E55 strict	E55-v2 strict
No multi-resource expansion UPA	0.333	0.217
No operation-mode binding UPA	0.148	0.217
No provenance overlay UPA	0.148	0.217
No alias resolution FDeny	0.211	0.238

Table 18: Human audit correction summary.

Quantity	Value
Human-audited rows	60
Decision agreement	54/60 = 0.900
All human checks true	46/60 = 0.767
Decision corrections	6
Atom corrections	13
Violation-reason corrections	10

B Audit Details

Leakage and label-hidden replay. The E55-v2 leakage audit reports `leakage_free=true` with no violations. E56 reports strict mode passed and label-hidden replay metrics equal for the original E55 contract. These checks support the claim that gold labels were not used as deployable inputs in the relevant evaluation path.

Reference authorizer. The E57-v2 reference authorizer is marked as an independent local implementation agreement check. It reports 600 rows, decision agreement 1.000, atom-count agreement 1.000, atom resource-set agreement 1.000, and no disagreements. Its claim scope is consistency checking, not independent deployment truth.

Resource perturbation. The E57-v2 resource-name perturbation keeps the authorization-aware guard’s UPA, coverage, decision accuracy, safe false-denial, and abstention unchanged. It reports changed decision count 0 and changed decision rate 0. This reduces the concern that the E55-v2 result depends only on memorized resource strings.

Human audit. The E57 human audit is a separate artifact. It reports 60 audited rows, 54/60 decision agreement, 46/60 all-checks-true, six decision corrections, thirteen atom corrections, and ten violation-reason corrections. The corrected cases primarily concern unknown-resource fallback and transaction-domain amount handling. The paper therefore uses corrected-label sensitivity wording.

C Reproduction Notes

Lightweight reproduction shim. The artifact includes two lightweight reproduction shims. The first command is:

```
python reproduce_main_numbers.py
```

Run it from this paper directory. It writes JSON, CSV, and Markdown outputs to the reproduction directory. It validates the main reported E48, E50, E55-v2, E55-v2 ablation, E56, E57-v2, and human-audit numbers and audit checks from fixed result artifacts.

The second command is:

```
python scripts/reproduce_all_main_tables.py
```

Run it from the artifact root. It validates Table 1, Tables 2–6, the E60–E64 tables, the E61 external-subset table, and the B8 released-adapter table from fixed result artifacts. It also writes all-main-table outputs and a reproduced claim-to-source map. Full experiment reruns are outside these lightweight checks. If a required artifact or JSON key is absent, the scripts exit nonzero and write a short error status.

For E60, the all-main-table shim depends on the independent-review validator output. A blocked strict-authorship status is not a reproduction failure; it is the conservative claim boundary used by the paper unless an external human review packet is provided and validated. For E61 external replay, the sidecar pipeline records the raw source hash, conversion rule, label source, and atom-sidecar source in the external raw manifest. For B8, the shim checks the comparable-adapter results under the label-hidden deployable-input contract.

Key source files. The main quantitative claims trace to:

- results/analysis/results/e48_tuple_guard_results.json
- results/analysis/results/e50_hard_guard_robustness_results.json
- audit/analysis/results/e55_v2_precommit_authz_results_strict.json
- results/analysis/results/e55_precommit_authz_results_strict.json
- results/analysis/results/e55_human_corrected_sensitivity.json
- audit/analysis/results/e56_final_report.json
- audit/analysis/results/e57_v2_validity_checks_report.json
- audit/analysis/results/e57_human_audit_validation.json
- evaluation/e60_heldout_contract/results_e60.json
- evaluation/e60_heldout_contract/e60_artifact_level_review.json
- evaluation/e61_realistic_trace_replay/results_e61.json
- evaluation/e61_realistic_trace_replay/external_trace_subset/raw_manifest.json
- evaluation/e61_realistic_trace_replay/external_trace_subset/results_external.json
- evaluation/e62_extraction_decomposition/
- evaluation/e63_interface_burden/
- baselines/baseline_results.json
- baselines/b8_released_guardrail/results_b8_*.json

Targeted tests from artifact notes. The existing material check reports that the following targeted test command passed in the artifact source root:

```
cd code
PYTHONPATH=. python -m pytest
../tests/tests/test_effect.binding_guard.e55.precommit_authz.py
../tests/tests/test_effect.binding_guard.e57.validity.py -q
```

with result 22 passed. This reproduction note reports fixed artifact checks; full experiment reruns are outside the lightweight number-validation path.

D Claim-to-Source Map

Table 19: Main claim-to-source map for the restructured v2 paper.

Claim	Source artifact	Safe wording
Side-effectful tool calls are containers of effect-resource-operation atoms.	E55 atom schema and expansion code	Atomization is the semantic object used by the local pre-commit mediator.
The paper is measurement plus diagnostic framework plus local pre-commit prototype.	Threat model, evaluation protocol, and limitation policy	Controlled custom-stress and local contract evidence, not a deployed-system guarantee.
The hard guard has bounded improvement.	E48 tuple guard; E50 robustness	Low but nonzero UPA/FDeny on E48 and a resource/auth bottleneck on E50.
Resource/auth is the central bottleneck.	E50 robustness	0.383 UPA at 0.921 coverage on controlled resource/auth stress.
Authorization-aware mediation improves local contract decidability.	E55-v2 strict	On E55-v2, coverage rises from 0.200 to 0.880 and UPA falls from 12/276 to 0/276.
Atom/authorization ablations identify which semantic interfaces carry the improvement.	E55-v2 strict ablations	Removing multi-resource, operation-mode, or provenance interfaces raises UPA; removing alias resolution raises FDeny.
Human-corrected sensitivity preserves zero UPA but introduces FDeny.	E55 human-corrected sensitivity	Corrected sensitivity has UPA 0/320 and FDeny 4/230.
Validity checks reduce specific artifact risks.	E57-v2 validity; E57 human audit	Perturbation/reference-authorizer checks pass; human audit finds corrections.
The atom interface transfers beyond E55-v2 under an independently specified contract.	E60 held-out contract manifest, leakage report, and results	E60 reaches 0.854 coverage, 0.000 UPA, and 0.812 atom exact-set match, with measurable FDeny and extraction degradation retained.
Atom-level mediation remains useful on sandboxed realistic trace replay.	E61 trace manifest, leakage report, and results	E61 reaches 0.847 coverage, 0.000 UPA, and 0.733 atom exact-set match; harder trace sources expose parsing and provenance failures.
Remaining errors are concentrated in extraction and missing/degraded authorization context.	E62 decomposition results	Gold atoms/context isolate checker behavior; extracted atoms and degraded context reduce atom match and coverage.
The authorization interface has measurable burden and degrades conservatively when incomplete.	E63 burden and context-degradation results	Missing, stale, or contradictory context usually increases abstention or FDeny rather than unsafe allow in the studied artifact.
Full atom-level mediation improves over non-atom local baselines, while diagnostic B5/B7 rows expose false-denial and capability-interface tradeoffs.	E64 baseline results	Baselines are local comparable adapters, not original-system benchmark reproductions; B7 can match full mediation when atom-equivalent capability context is supplied.